

Basi di Dati - Modulo 1

Sistema Informativo

Immagina un'organizzazione (che può essere un'azienda, una scuola o qualsiasi altra realtà) come una macchina che funziona grazie alle informazioni. Un sistema informativo è ciò che permette di gestire queste informazioni: raccoglierle, organizzarle, conservarle e farle circolare all'interno della struttura.

Un Sistema Informativo ha dunque il compito di gestire le informazioni, rendendole disponibili per altre funzioni dell'organizzazione

In passato, ogni programma o applicazione aveva i propri file, ognuno separato dagli altri. Ad esempio, se un ufficio aveva un programma per gestire i clienti e un altro per gestire i dipendenti, questi due programmi non condividevano le informazioni tra loro. Ad oggi possiamo chiaramente dire che questa non è la soluzione migliore dal momento che introduceva una serie di problemi come:

- **Ridondanza:** se un'informazione serviva a due programmi, dovevi salvarla due volte, portando inevitabilmente a duplicati.
- **Inconsistenza:** se aggiornavi un dato in un programma, l'altro non veniva aggiornato automaticamente, ma andava fatto a mano, creando confusione e probabili errori di distrazione.
- **Dipendenza dai programmi:** ogni programma gestiva i dati a modo suo, rendendo difficile la manutenzione e l'aggiornamento delle informazioni.

Attualmente invece i Sistemi Informativi fanno ampio uso di Database e Data Base Management System (anche detti BDMS) per risolvere i problemi sopra citati. Un Database è un insieme organizzato di dati che possono essere acceduti e gestiti in maniere efficiente tramite i cosiddetti Database Management System o DBMS.

In generale i dati contenuti all'interno di un Sistema Informativo possono essere classificati come Strutturati o Non: I **Dati Strutturati** sono informazioni che sono ben organizzate, come tabelle con nomi, date, numeri, mentre quelli **Non Strutturati** sono dati senza un'organizzazione precisa e spesso scritti in linguaggio naturale.

I database consentono anche la **Condivisione delle Informazioni** tra più componenti del sistema informativo (per es. due reparti di uno stesso negozio) riducendo ridondanze e inconsistenze. Tuttavia questo richiede una certa attenzione nella gestione della privacy, per permettere solo a chi è autorizzato di vedere e modificare un determinato campo gestendo anche i casi di concorrenza in cui due soggetti accedono allo stesso dato contemporaneamente.

Livelli di Astrazione

Un database non mostra tutte le informazioni in modo diretto. Per rendere il tutto più comprensibile e organizzato, si divide la struttura in tre livelli principali:

1. **Schema esterno:** Questo è quello che un utente vede. È una versione "semplificata" o "filtrata" del database. Ogni utente può vedere solo le informazioni che gli interessano o che ha il permesso di vedere. Ad esempio, in una scuola gli studenti potrebbero non vedere l'indirizzo di casa dei professori.
2. **Schema logico:** È la struttura completa del database, come viene organizzato internamente. Qui si definiscono le tabelle, le relazioni tra i dati ma non si entra nel come i dati vengono fisicamente memorizzati.
3. **Schema fisico:** Questo livello descrive come i dati sono effettivamente memorizzati nei file fisici. Il DBMS si occupa di tutto questo, quindi l'utente non deve preoccuparsi di dove e come i dati sono salvati, ma solo di usarli.

Inoltre, un altro aspetto fondamentale di un DBMS è l'**indipendenza dei dati**, che si divide in due parti:

- **Indipendenza fisica**: è possibile cambiare il modo in cui i dati sono salvati fisicamente (per esempio, spostarli su un altro server), senza dover cambiare il modo in cui gli utenti li vedono o li usano.
- **Indipendenza logica**: è possibile fare modifiche alla struttura interna del database (ad esempio, aggiungere nuovi campi alle tabelle), senza cambiare le informazioni visibili agli utenti.

Schemi e istanze

In ogni base di dati esistono:

- **Lo schema** è come un disegno o un modello fisso che descrive la struttura del database. Esso è sostanzialmente invariante nel tempo (es. le intestazioni delle tabelle, tipo nome – cognome – indirizzo etc).
- **L'istanza** è il contenuto attualmente presente nelle tabelle (es i dati concreti, come Francesco – Giammaria – Via Antonio Scarpa) e può cambiare molto rapidamente anche solo aggiungendo/rimuovendo altre istanze.

NOME	COGNOME	DATAN	LUOGON	SCHEMA
Piero	Napoli	22-10-63	Bari	ISTANZE
Marco	Bianchi	01-05-54	Roma	
Maria	Rossi	09-02-68	Milano	
Maria	Bianchi	07-12-70	Bari	
Paolo	Sossi	15-03-75	Palermo	

Linguaggi di interazione

I tipi di linguaggi principali che consentono di interagire/lavorare con i database sono due:

- **DDL (Data Definition Language)**: Servono per definire la struttura del database, ad esempio per creare o modificare le tabelle.
- **DML (Data Manipulation Language)**: Serve per leggere o modificare i dati.

Modelli di Database

Nel tempo, sono stati sviluppati diversi **Modelli** per organizzare i dati all'interno dei database. Questi modelli definiscono come i dati sono collegati tra loro e come possono essere consultati. Ecco i principali:

- **Modello Reticolare**: I dati sono organizzati in una rete di record collegati tramite puntatori.
- **Modello Gerarchico**: I dati sono organizzati in modo più rigido, come un albero. Ogni dato ha un "genitore" (come una cartella che contiene altre cartelle).
- **Modello Relazionale**: Organizza i dati in tabelle di record omogenei (es. studenti, corsi, esami). Non utilizza puntatori espliciti, semplificando la rappresentazione dei dati.
- **Modello a Oggetti**: Introduce la nozione di oggetti, che contengono sia dati (attributi) che comportamenti (metodi). Questo modello è stato introdotto negli anni '80 e non ha ancora uno standard universale.

Modello Relazionale

Il **Modello Relazionale** è un modo di organizzare i dati nei database, proposto nel 1970 da uno studioso di nome E. F. Codd. Questo modello è stato creato per rendere più facile lavorare con i dati in modo indipendente dalla struttura fisica del database (come sono memorizzati i dati "dietro le quinte"). Però ci sono voluti più di dieci anni prima che fosse implementato in sistemi reali, perché costruire database efficienti e affidabili non è semplice.

Il modello relazionale **si basa sul concetto matematico di "relazione"**. Nel contesto dei database, una relazione è molto simile a una tabella, quindi quando parliamo di "relazione" in questo ambito, in realtà stiamo parlando di tabelle, come quelle di Excel. Queste tabelle sono composte da righe e colonne, dove ogni colonna contiene un tipo specifico di dati (ad esempio nomi, cognomi, indirizzi, ecc.), e ogni riga, che viene detta tupla, contiene una serie di dati relative ad una specifica entità (come una persona, un prodotto, un esame etc)

IMPORTANTE! Una Relazione nel modello concettuale Entità-Relazioni viene tradotta anche con associazione perché rappresenta appunto un tipo di collegamento (relazione) concettuale tra entità diverse (ad esempio Esame mette in relazione uno Studente ed un Insegnamento) ... ma questo sarà trattato nel secondo modulo del corso.

Definizioni

Dominio: l'insieme, possibilmente infinito, di appartenenza di un certo tipo di dati (es. l'insieme dei numeri interi, l'insieme dei numeri decimali, l'insieme delle stringhe di lunghezza = 20, l'insieme {0,1} e così via). Quasi sempre si usano domini predefiniti, comuni ai linguaggi di programmazione

Relazione Matematica: è un qualsiasi sottoinsieme del prodotto cartesiano di uno o più domini. Una relazione che è sottoinsieme del prodotto Cartesiano di k domini si dice di grado k.

Gli elementi di una relazione sono detti tuple (oppure n-uple o ennuple). Il numero di tuple di una relazione è la sua cardinalità. Ogni tupla di una relazione di grado k ha k componenti ordinate ma non c'è ordinamento tra le tuple ed esse sono tutte distinte (almeno per un valore).

Esempio:

- Supponiamo $k = 2$

Numero dei Domini

- $D1 = \{\text{bianco, nero}\}, D2 = \{0, 1, 2\}$

Domini

$D1 \times D2 = \{(\text{bianco}, 0), (\text{bianco}, 1), (\text{bianco}, 2), (\text{nero}, 0), (\text{nero}, 1), (\text{nero}, 2)\}$

Prodotto Cartesiano

$\{(\text{bianco}, 0), (\text{nero}, 0), (\text{nero}, 2)\}$ è una relazione di grado 2, cardinalità 3 e con tuple (bianco, 0), (nero, 0), (nero, 2)

$\{(\text{nero}, 0), (\text{nero}, 2)\}$ è una relazione di grado 2, cardinalità 2 e con tuple (nero, 0), (nero, 2)

Notazione

- Se r è una relazione di grado k
- se t è una tupla di r
- se i è un numero intero nell'insieme $\{1, \dots, k\}$
 $t[i]$ (oppure $t.i$) indica la i -sima componente di t

- esempio:

- se $r = \{(0,a), (0,c), (1,b)\}$

- $t = (0,a)$ è una tupla di r

- $t[2] = a$

- $t[1] = 0$

- $t[1,2] = (0, a)$



0	a
0	c
1	b

Relazioni e Tabelle



Ricapitolando:

Una relazione può essere implementata come una tabella in cui ogni riga è una tupla distinta della relazione e ogni colonna corrisponde ad una componente (valori omogenei, cioè provenienti dallo stesso dominio).

Le colonne corrispondono ai domini $D_1, D_2, \dots, D_k$ e hanno associati dei nomi univoci all'interno della tabella, detti nomi degli attributi (descrivono il ruolo, come nome, cognome, indirizzo etc)

La coppia (nome di attributo, dominio) è chiamata attributo. L'insieme di attributi di una relazione è detto schema.

Se una relazione è denominata R e i suoi attributi hanno nomi $A_1, A_2, \dots, A_k$, lo schema è spesso indicato da:

$R(A_1, A_2, \dots, A_k)$

Nell'ultima definizione di modello relazionale, le componenti di una relazione sono indicate dai nomi degli attributi, anziché dalla posizione $t[A_i]$ indica il valore dell'attributo con nome A_i della tupla t

Vincoli di Integrità

In alcuni casi le Basi di Dati vengono definite come “Scorrette” se presentano degli errori o delle incongruenze tra i dati. Esempio:

IMPIEGATO CODICE	COGNOME	NOME	RUOLO	ASSUNZIONE	DIP
COD1	Rossi	Mario	Analista	1795	01
COD2	Bianchi	Pietro	Analista	1990	05
COD2	Neri	Paolo	Amministratore	1985	01

DIPARTIMENTO NUMERO	NOME
01	Progettazione
02	Amministrazione

COD2 Duplicato...

Data di Assunzione impossibile

Codice dipartimento inesistente

Oppure

Esami	Studente	Voto	Lode	Corso
	276545	32		01
	276545	30	si	02
	787643	27	si	03
	739430	15		04

Studenti	Matricola	Cognome	Nome
	276545	Rossi	Mario
	787643	Neri	Piero
	787643	Bianchi	Luca

Studenti con la stessa matricola

Voti sotto il 18 e superiori al 30

27 con lode....

Tutti queste incongruenze possono essere risolte, o evitate, inserendo dei **Vincolo di integrità**, cioè delle proprietà che deve essere soddisfatta da ogni istanza della base di dati. **Un'istanza di base di dati è corretta se soddisfa tutti i vincoli di integrità associati al suo schema.**

Vincoli Intra-relazionali

Vincolo di chiave primaria (primary key): unica e mai nulla

Vincoli di dominio (es. voto ≥ 18 AND voto ≤ 30)

Vincoli di unicità (unique)

Vincoli di esistenza per un certo attributo (not null)

Espressioni sul valore di attributi della stessa tupla (es. data_arrivo < data_partenza)

Vincoli Inter-relazionali

Questi vincoli sono presenti in tabelle diverse (o relazioni). Il loro scopo principale è garantire la **consistenza** e l'**integrità** dei dati, assicurandosi che le informazioni contenute in tabelle collegate siano corrette e coerenti. Un esempio comune di vincolo inter-relazionale è la **chiave esterna** (foreign key), che permette di collegare una tabella a un'altra attraverso un campo che fa riferimento alla chiave primaria dell'altra tabella.

Ad esempio, il **vincolo di integrità referenziale** garantisce che un valore nella colonna di chiave esterna di una tabella corrisponda sempre a un valore valido nella colonna di chiave primaria della tabella a cui è collegato. Questo impedisce, ad esempio, di avere un esame associato a uno studente inesistente.

Chiavi

Quando abbiamo una tabella (o relazione) in un database, ogni riga (o tupla) rappresenta un insieme di dati. Per poter distinguere una riga dall'altra in modo univoco, abbiamo bisogno di un metodo che permetta di identificare ciascuna riga in maniera unica. Qui entrano in gioco le **chiavi**.

def: Una **chiave** è un attributo o un insieme di attributi che permette di identificare **univocamente** una tupla.

Un insieme di attributi è considerato una chiave se rispetta due condizioni:

1. **Unicità:** Per ogni riga della tabella, non possono esserci due tuple con gli stessi valori per gli attributi che formano la chiave. In altre parole, ogni riga deve essere unica rispetto alla chiave.

Esempio: Se "ID Studente" è una chiave, non ci saranno due studenti con lo stesso ID.

2. **Minimo insieme di attributi:** Nessun sottoinsieme della chiave può soddisfare la prima condizione. Questo significa che la chiave deve essere composta dal minor numero di attributi possibile per mantenere l'unicità.

Esempio: Se la chiave è formata da "Nome" e "Cognome", ma "ID Studente" da solo è sufficiente per garantire l'unicità, allora "ID Studente" è preferito come chiave.

A volte, una tabella (o relazione) può avere più di una possibile chiave. Queste vengono chiamate **chiavi alternative**. Tuttavia, quando dobbiamo scegliere una chiave da usare abitualmente, preferiamo quella più semplice o quella che utilizza meno attributi, chiamata **chiave primaria**, la quale deve avere alcune caratteristiche speciali:

- **Non ammette valori nulli:** Ogni riga deve avere un valore valido per la chiave primaria. Non possono esistere righe con valori nulli in una chiave primaria.
- **Deve esistere sempre una chiave:** Per garantire che non ci siano due righe uguali in una tabella, deve esserci almeno una chiave che distingue ogni riga.

Dipendenze Funzionali

Una dipendenza funzionale stabilisce un particolare legame semantico tra due insiemi. In pratica, una dipendenza funzionale dice che:

Se conosciamo il valore di un attributo X possiamo determinare univocamente il valore di un altro attributo Y

Questa relazione è scritta come $X \rightarrow Y$ e si legge **X DETERMINA Y**.

Esempio: Supponiamo di avere uno schema di relazione VOLI (CodiceVolo, Giorno, Pilota, Ora). I vincoli corrispondono alle dipendenze funzionali.

- $\text{CodiceVolo} \rightarrow \text{Ora}$ *(Se conosci il CodiceVolo, sai esattamente l'Ora di partenza)*
- $\{\text{Giorno}, \text{Pilota}, \text{Ora}\} \rightarrow \text{CodiceVolo}$ *(Se conosci il Giorno, il pilota e l'ora del volo puoi ricavare il codice del volo)*
- $\{\text{CodiceVolo}, \text{Giorno}\} \rightarrow \text{Pilota}$ *(Se conosci il CodiceVolo ed il giorno puoi ricavare chi è il pilota)*

Algebra Relazionale

Questa parte sarà fondamentale per il primo esercizio della prima parte dell'esame scritto.

L'algebra relazionale è un sistema di calcolo che ci consente di interrogare le nostre istanze. È un linguaggio formale perché contiene una serie di operatori che, in maniera procedurale (cioè passo passo), ci consente di interrogare la base dati. Gli operatori sono per la maggior parte binari, sebbene esistano una serie di operatori unari. Tutti questi operatori forniscono come risultato una nuova istanza di relazione (che può essere «salvata» in una «variabile»).

Proiezione

È l'operatore unario più semplice. Si indica con il simbolo di pi greco e consente di effettuare un "taglio verticale" su una relazione, restituendo le colonne che corrispondono ad un determinato attributo. Esempio:

Cliente

Nome	C#	Città
Rossi	C1	Roma
Rossi	C2	Milano
Bianchi	C3	Roma
Verdi	C4	Roma

Query: Nomi dei clienti

$\pi_{\text{Nome}}(\text{Cliente})$

Rossi
Bianchi
Verdi

ATTENZIONE: Si seguono le regole insiemistiche
Nella relazione risultato NON ci sono DUPLICATI

Si noti l'assenza di duplicati. Infatti questo operatore funziona in maniera "insiemistica" e quindi nomi uguali vengono rimossi. Per evitare ciò si può chiamare l'operatore sulla chiave, che è univoca per ogni tupla:

Cliente

Nome	C#	Città	
Rossi	C1	Roma	
Rossi	C2	Milano	
Rossi	C3	Roma	
Bianchi	C4	Roma	
Verdi	C5	Roma	

Query: Nomi e codici dei clienti

$\pi_{\text{Nome,C\#}}(\text{Cliente})$

Rossi	C1
Rossi	C2
Rossi	C3
Bianchi	C4
Verdi	C5

Selezione

Consente di effettuare un “taglio orizzontale” su una relazione, cioè di selezionare solo le righe (tuple) che soddisfano una data condizione. Si denota con il simbolo σ (sigma)

La condizione di selezione è un’espressione booleana composta (tramite operatori AND, OR e NOT) in cui i termini semplici sono del tipo: $A\theta B$ oppure $A\theta a$ dove $\theta \in \{<, =, >, \leq, \geq\}$

Esempio:

Cliente	Nome	C#	Città	
	Rossi	C1	Roma	
	Rossi	C2	Milano	
	Rossi	C3	Roma	
	Bianchi	C4	Roma	
	Verdi	C5	Roma	

Query: Dati dei clienti che risiedono a Roma

$\sigma_{\text{Città}='Roma'}(\text{Cliente})$

Rossi	C1	Roma	
Rossi	C3	Roma	
Bianchi	C4	Roma	
Verdi	C5	Roma	

02/10/2024

Unione

Consente di costruire una relazione contenente tutte le ennuple che appartengono ad **almeno uno** dei due operandi.

Si denota con il simbolo di U. L’operazione di unione può essere applicata solo a operandi **union compatibili** cioè tali che:

- hanno lo stesso numero di attributi
- gli attributi corrispondenti dominio (nell’ordine) sono definiti sullo stesso

NOTA: NON è necessario che gli attributi abbiano lo stesso nome, ma ovviamente il risultato ha senso se hanno un significato omogeneo. Per esempio Numero di esami ed Età potrebbero essere definiti sullo stesso dominio, ma non avrebbe senso unire due relazioni che hanno questi attributi in colonne corrispondenti.

Docenti	Nome	CodDoc	Dipartimento	Amministrativi	Nome	CodAmm	Dipartimento
	Rossi	C1	Matematica		Esposito	C1	Lingue
	Rossi	C2	Lettere		Riccio	C2	Matematica
	Bianchi	C3	Matematica		Pierro	C3	Lettere
	Verdi	C4	Lingue		Bianchi	C4	Lingue

$$\text{Personale} = \text{Docenti} \cup \text{Amministrativi}$$

Nome	Cod	Dipartimento
Rossi	C1	Matematica
Rossi	C2	Lettere
Bianchi	C3	Matematica
Verdi	C4	Lingue
Esposito	C1	Lingue
Riccio	C2	Matematica
Pierro	C3	Lettere
Bianchi	C5	Lingue

Ma se le due Relazioni NON fosse Union Compatibili?

Docenti	Nome	CodDoc	Dipartimento
	Rossi	D1	Matematica
	Rossi	D2	Lettere
	Bianchi	D3	Matematica
	Verdi	D4	Lingue

Amministrativi	Nome	CodAmm	Dipartimento	Stip
	Esposito	A1	Lingue	1250
	Riccio	A2	Matematica	2000
	Pierro	A3	Lettere	1000

In questo esempio non è possibile unire le due relazioni in quanto non sono union compatibili: hanno un numero diverso di attributi!! Quello che possiamo fare è una Proiezione su entrambe le relazioni andando così a “tagliare via” gli attributi che ci impediscono di fare l’unione:

$$\text{Personale} = \pi_{\text{Nome, CodDoc, Dipartimento}}(\text{Docenti}) \cup \pi_{\text{Nome, CodAmm, Dipartimento}}(\text{Amministrativi})$$

E stessa cosa vale se ho problemi di Dominio. Esempio:

Docenti	Nome	CodDoc	Dipartimento
	Rossi	D1	Matematica
	Rossi	D2	Lettere
	Bianchi	D3	Matematica
	Verdi	D4	Lingue

Amministrativi	Nome	CodAmm	Dipartimento	Stip
	Esposito	A1	Lingue	1250
	Riccio	A2	Matematica	2000
	Pierro	A3	Lettere	1000

$$\text{Personale} = \pi_{\text{Nome, CodDoc}}(\text{Docenti}) \cup \pi_{\text{Nome, CodAmm}}(\text{Amministrativi})$$

Differenza

Costruisce una relazione contenente tutte le tuple che appartengono al primo operando E NON al secondo operando. Si denota con il simbolo “-”. Come prima, si applica solo ad operandi **union compatibili**.

Studenti	Nome	CodFiscale	Dipartimento	Amministrativi	Nome	CodFiscale	Dipartimento
	Rossi	C1	Matematica		Esposito	C5	Lettere
	Rossi	C2	Lettere		Riccio	C6	Matematica
	Bianchi	C3	Matematica		Pierro	C7	Lingue
	Verdi	C4	Lingue		Bianchi	C3	Matematica

ATTENZIONE! La Differenza NON è commutativa!!

- Studenti – Amministrativi = studenti che non sono amministrativi

Studenti	Nome	CodFiscale	Dipartimento
	Rossi	C1	Matematica
	Rossi	C2	Lettere
	Verdi	C4	Lingue

- Amministrativi – Studenti = amministrativi che non sono studenti

Amministrativi	Nome	CodFiscale	Dipartimento
	Esposito	C5	Lettere
	Riccio	C6	Matematica
	Pierro	C7	Lingue

Intersezione

Consente di costruire una relazione contenente tutte le tuple che **appartengono ad entrambi gli operandi**. Si denota con il simbolo $\cap$. Si applica a operandi **union compatibili**.

Studenti	Nome	CodFiscale	Dipartimento	Amministrativi	Nome	CodFiscale	Dipartimento
	Rossi	C1	Matematica		Esposito	C5	Lettere
	Rossi	C2	Lettere		Riccio	C6	Matematica
	Bianchi	C3	Matematica		Pierro	C7	Lingue
	Verdi	C4	Lingue		Bianchi	C3	Matematica

$\text{Studenti} \cap \text{Amministrativi} = \text{studenti che sono anche amministrativi}$

Studenti Amm	Nome	CodFiscale	Dipartimento
	Bianchi	C3	Matematica

Prodotto Cartesiano

Consente di costruire una relazione contenente tutte le ennuple che si ottengono concatenando una ennupla del primo operando con una ennupla del secondo operando. Si denota con il simbolo "x" e si usa quando le informazioni che occorrono a rispondere ad una query si trovano in relazioni diverse.

Capita molto spesso che le informazioni che interessano per rispondere ad una interrogazione siano distribuite in più relazioni, in quanto coinvolgono più oggetti in qualche modo associati. Vedremo infatti più avanti che per garantire determinate «buone» qualità di una relazione occorre rappresentare separatamente, in relazioni diverse, dei concetti.

Cliente	Nome	C#	Città	Ordine	O#	C#	A#	N-pezzi
	Rossi	C1	Roma		O1	C1	A1	100
	Rossi	C2	Milano		O2	C2	A2	200
	Bianchi	C3	Roma		O3	C3	A2	150
	Verdi	C4	Roma		O4	C4	A3	200
					O1	C1	A2	200
					O1	C1	A3	100

Query: Restituire i dati dei clienti e degli ordini (ClientexOrdine)

Per poter distinguere gli attributi con lo stesso nome nello schema risultante possiamo usare l'operazione di ridenominazione(ρ) per utilizzare una copia della relazione. In questo caso abbiamo il campo C# duplicato, e dunque:

1. **OrdineR= $\rho_{CC\#} \leftarrow C\#(\text{Ordine})$** : Crea una nuova variabile relazionale OrdineR in cui il campo C# è rinominato in CC#

Risultato del Prodotto Cartesiano

Nome	C#	Città	O#	CC#	A#	N-pezzi
Rossi	C1	Roma	O1	C1	A1	100
Rossi	C1	Roma	O2	C2	A2	200
Rossi	C1	Roma		C3	A2	150
Rossi	C1	Roma		C4	A3	200
Rossi	C1	Roma		C1	A2	200
Rossi	C2	Milano		C1	A1	100
...	...	...		...	...	...
Bianchi	C3	Roma		C1	A1	100
...	...	...		...	...	...
Verdi	C4	Roma			A3	200
...	...	...		...	...	...

Tuttavia il prodotto cartesiano ci restituisce TUTTE le coppie di elementi delle due relazioni. Anche quelle “inutili” o incongruenti (come quella evidenziata in rosso), in cui i codici clienti non corrispondono...

Per risolvere questo problema possiamo tranquillamente utilizzare la selezione per andare a prendere solo le coppie in cui C# = CC#. Cioè solo quelle in cui il codice cliente corrisponde con il cliente effettivo che ha acquistato.

Query Corretta:

$$\sigma_{C\#=CC\#}(Cliente \times OrdineR)$$

Nome	C#	Città	O#	CC#	A#	N-pezzi
Rossi	C1	Roma	O1	C1	A1	100
Rossi	C1	Roma	O1	C1	A2	200
Rossi	C1	Roma	O1	C1	A3	100
Rossi	C2	Milano	O2	C2	A2	200
Bianchi	C3	Roma	O3	C3	A2	150
Verdi	C4	Roma	O4	C4	A3	200

Una query più «elegante»

Per migliorare la query possiamo eliminare gli attributi duplicati (che di solito sono proprio quelli della condizione di selezione). Per fare ciò attuiamo una proiezione che prende solo le colonne “distinte”:

$$\pi_{\text{Nome C\# Città O\# A\# N-pezzi}}(\sigma_{C\#=CC\#}(\text{Cliente} \times \text{OrdineR}))$$

Nome	C#	Città	O#	A#	N-pezzi
Rossi	C1	Roma	O1	A1	100
Rossi	C1	Roma	O1	A2	200
Rossi	C1	Roma	O1	A3	100
Rossi	C2	Milano	O2	A2	200
Bianchi	C3	Roma	O3	A2	150
Verdi	C4	Roma	O4	A3	200

Tutto questo procedimento, compresa la rimozione delle colonne duplicate, si può effettuare utilizzando un altro operatore apposito, chiamato **Join Naturale**...

Join Naturale

Consente di selezionare le tuple del prodotto cartesiano dei due operandi che soddisfano la condizione:

$$R1.A1 = R2.A1 \wedge R1.A2 = R2.A2 \wedge \dots \wedge R1.Ak = R2.Ak$$

(dove R1 ed R2 sono i nomi delle relazioni operando e A1,,A2,...,Ak sono gli attributi comuni, cioè con lo stesso nome, delle relazioni operando) eliminando le ripetizioni degli attributi.

$$r1 \bowtie r2 = \pi_{xy}(\sigma_C(r1 \times r2))$$

dove:

- C: $R1.A1 = R2.A1 \wedge \dots \wedge R1.Ak = R2.Ak$
- Xnè l'insieme di attributi di r1
- Y è l'insieme di attributi di r2 che non sono attributi di r1

DA RICORDARE: nel join naturale gli attributi della condizione che consente di unire solo le ennuple giuste hanno lo stesso nome, vengono unite le ennuple in cui questi attributi hanno lo stesso valore

Cliente	Nome	C#	Città
	Rossi	C1	Roma
	Rossi	C2	Milano
	Bianchi	C3	Roma
	Verdi	C4	Roma

Ordine	O#	C#	A#	N-pezzi
	O1	C1	A1	100
	O2	C2	A2	200
	O3	C3	A2	150
	O4	C4	A3	200
	O1	C1	A2	200
	O1	C1	A3	100

Query: Dati dei clienti e dei loro ordini

Cliente $\bowtie$ *Ordine*

Nome	C#	Città	O#	A#	N-pezzi
Rossi	C1	Roma	O1	A1	100
Rossi	C1	Roma	O1	A2	200
Rossi	C1	Roma	O1	A3	100
Rossi	C2	Milano	O2	A2	200
Bianchi	C3	Roma	O3	A2	150
Verdi	C4	Roma	O4	A3	200

Il Join Naturale di un'istanza con sé stessa restituisce l'istanza stessa!!!

Esempio2: Nomi e città dei clienti che hanno ordinato più di 100 pezzi per almeno un articolo con prezzo superiore a 2

Cliente	Nome	C#	Città	Ordine	O#	C#	A#	N-pezzi	Articolo	A#	Denom.	Prezzo
	Rossi	C1	Roma		O1	C1	A1	100		A1	Piatto	3
	Rossi	C2	Milano		O2	C2	A2	200		A2	Bicchiere	2
	Bianchi	C3	Roma		O3	C3	A2	150		A3	Tazza	4
	Verdi	C4	Roma		O4	C4	A3	200				
					O1	C1	A2	200				
					O1	C1	A3	100				

Risultato del Join Naturale:

(Cliente Ordine)>< Articolo)

Nome	C#	Città	O#	A#	N-pezzi	Denom.	Prezzo
Rossi	C1	Roma	O1	A1	100	Piatto	3
Rossi	C1	Roma	O1	A2	200	Bicchiere	2
Rossi	C1	Roma	O1	A3	100	Tazza	4
Rossi	C2	Milano	O2	A2	200	Bicchiere	2
Bianchi	C3	Roma	O3	A2	150	Bicchiere	2
Verdi	C4	Roma	O4	A3	200	Tazza	4

Risultato dopo aver inserito le condizioni:

$\sigma_{N-pezzi>100 \wedge Prezzo>2}((\text{Cliente Ordine})>< \text{Articolo})$

Nome	C#	Città	O#	A#	N-pezzi	Denom.	Prezzo
Verdi	C4	Roma	O4	A3	200	Tazza	4

Risultato dopo aver proiettato solo i dati che ci sono richiesti:

$\pi_{Nome,Città}(\sigma_{N-pezzi>100 \wedge Prezzo>2}((\text{Cliente Ordine})>< \text{Articolo}))$

Nome	Città
Verdi	Roma

Esempio 2 Alternativa

Effettuando prima la selezione delle tuple che ci interessano, l'operazione è più efficiente perché evitiamo di combinare inutilmente dati che poi non ci serviranno.

O#	C#	A#	N-pezzi
O2	C2	A2	200
O3	C3	A2	150
O4	C4	A3	200
O1	C1	A2	200

$\sigma_{N-pezzi > 100}(\text{Ordine})$

Nome	C#	Città	O#	A#	N-pezzi
Rossi	C1	Roma	O1	A2	200
Rossi	C2	Milano	O2	A2	200
Bianchi	C3	Roma	O3	A2	150
Verdi	C4	Roma	O4	A3	200

Cliente $\bowtie \sigma_{N-pezzi > 100}(\text{Ordine})$

Theta Join

$$r_1 \bowtie_{A\theta B} r_2 = \sigma_{A\theta B}(r_1 \times r_2)$$

Consente di selezionare le tuple del prodotto cartesiano dei due operandi che soddisfano una condizione $A\theta B$, dove:

- θ è un operatore di confronto ($\theta \in \{, =, <, >, \leq, \geq\}$)
- A è un attributo dello schema del primo operando
- B è un attributo dello schema del secondo operando
- $\text{dom}(A) = \text{dom}(B)$ (Altrimenti non potremmo utilizzare gli operatori di confronto...)

Esempio: Il theta join lo possiamo utilizzare anche per unire una tabella con sé stessa! Nell'esempio qui sotto infatti abbiamo che ogni cliente ha associato il suo capo. Tuttavia non esiste una relazione capo separata perché anche il capo è un impiegato. E allora come facciamo a trovare, per ogni impiegato, il suo capo?

Impiegati

Nome	C#	Dipart	Stip	Capo#
Rossi	C1	B	100	C3
Pirlo	C2	A	200	C3
Bianchi	C3	A	500	NULL
Verdi	C4	B	200	C2
Neri	C5	B	150	C1
Tosi	C6	B	100	C1

Impiegati $\bowtie$ Impiegati
C# = Capo#

Condizioni che implicano il quantificatore universale

Fino ad ora abbiamo visto query che implicavano condizioni equivalenti al quantificatore esistenziale $\exists$. Il meccanismo di valutazione sequenziale dell'algebra relazionale consente di rispondere facilmente a questo tipo di query.

Infatti in qualunque posizione appaiono nell'espressione di algebra relazionale, la valutazione delle condizioni avviene in sequenza, tupla per tupla, e quando si incontra una tupla che soddisfa le condizioni, questa viene inserita nel risultato (eventualmente parziale)

La condizione potrebbe però richiedere la valutazione di gruppi interi di tuple prima di decidere se inserirle tutte, qualcuna o nessuna. In questo caso la condizione equivale a valutare il quantificatore universale $\forall$ oppure $\neg \exists$.

Il problema principale nella realizzazione di queste query sta nel fatto che le tuple non sono ordinate e la valutazione delle condizioni avviene in sequenza, tupla per tupla, e una volta inserita una tupla nel risultato non possiamo più eliminarla.

In questi casi quello che si può fare è "pensare al contrario" ... cioè: invece di cercare quali soddisfano la proprietà cerchiamo quelli che NON soddisfano la proprietà e li sottraiamo alla relazione iniziale. Stiamo quindi calcolando il cosiddetto complementare.